

1. Установка ROS2 Humble + Gazebo :

если не установлен ROS2 нативно, проще всего его установить в докере:

```
docker pull osrf/ros:humble-desktop-full
```

<https://docs.ros.org/en/humble/Tutorials/Advanced/Simulators/Gazebo/Gazebo.html>

2. Запуск Gazebo

В ROS2 Gazebo запускается командой `ign`:

```
ign gazebo -v 4 -r visualize_lidar.sdf
```

Откроется окошко с газебовским миром и моделькой робота из шариков и параллелепипедов. Моделька робота имеет лидар, но топики с него сразу не видны в ROS - нужен `ros_gz_bridge`. Свяжем ROS и Gazebo согласно <https://docs.ros.org/en/humble/Tutorials/Advanced/Simulators/Gazebo/Gazebo.html>

3. Запуск Gazebo с моделькой джетбота на складе

Сначала просто запустим Gazebo:

```
ros2 launch ros_gz_sim gz_sim.launch.py
```

Если долго не запускается – можно закрыть и запустить еще раз, вполне возможно что запустится. Далее в открывшемся окошке выбираем “Tugbot in warehouse” – загрузится среда с моделью склада и моделью робота Tugbot на складе.

Вместо робота Tugbot хотелось бы видеть модельку нашего Jetbot-a

Она есть в https://github.com/KirillMouraviev/jetbot_ros_gazebo Репозиторий нужно клонировать и собрать в воркспейсе как ROS2 пакет.

Затем нужно открыть файл `/root/.ignition/fuel/fuel.gazebosim.org/openrobotics/worlds/tugbot\ in\ warehouse/2/tugbot_warehouse.sdf` (например, с помощью расширения Docker-a для vscode) и вместо модельки Tugbot-a поставить туда ссылку на модельку Jetbot-a и поменять позицию на `'''<pose>12.0 -15.0 0.05 -1.5707 0 0</pose>'''` - позиция между маленькими полками и поворот модели на $\pi/2$, а также поменять имя ``tugbot`` на ``jetbot`` - иначе в следующих разделах мы делаем бриджи с газебо в ros2 (там уже все готово) - они мапят по имени `jetbot`, можно оставить `tugbot`, но тогда придется в репозитории в бриджах менять `jetbot` на `tugbot`

Далее снова в папке воркспейса (там где делался `git clone`) - `'''colcon build --symlink-install'''`, `'''source install/setup.bash'''`

После встраивания модельки джетбота запустим Gazebo со средой Tugbot in warehouse:

```
ign gazebo -v 4 -r /root/.ignition/fuel/fuel.gazebosim.org/openrobotics/worlds/tugbot\ in\ warehouse/2/tugbot_warehouse.sdf
```

4. Прокидывание топиков с модельки джетбота

Для запуска навигационного стека нам нужны будут следующие данные:

- Скан с лидара
- Картинка с камеры

- Одометрия

Также нужен будет контроллер для движения робота.

Скан с лидара и картинка с камеры публикуются симулятором, т.к. в модельке прописаны сенсоры. Но по умолчанию у них в топиках будет некорректный `frame_id`. Для корректных `frame_id` (`lidar_link` и `camera_link` соответственно) в описания сенсоров нужно вставить вот такую строчку:

```
<ignition_frame_id>lidar_link</ignition_frame_id>
<ignition_frame_id>camera_link</ignition_frame_id>
```

Чтобы не прописывать огромную команду `ros2 run` для запуска всех бриджей, объединим их в один лаунчфайл. Можно это сделать, например, <https://chat.deepseek.com/share/lmknmp64bpzxazgqgu>. Сохраним лаунчфайл в любом `ros2` пакете и запустим его.

5. Создание дерева трансформов и проверка конфигурации в RViz

Для корректной работы навигационного стека, помимо данных с сенсоров в топиках, нужны корректные трансформы – преобразования между системами координат (фреймами), связанными с началом отсчета одометрии, центром робота, его сенсорами и т.д. Для задания статических трансформов (между различными деталями робота) добавим в `sdf`-файл с моделькой плагин `JointStatePublisher`:

```
<plugin filename="ignition-gazebo-joint-state-publisher-system"
        name="ignition::gazebo::systems::JointStatePublisher">
    <topic>/joint_state</topic>
</plugin>
```

Он публикует сообщения особого типа – `sensor_msgs/msg/JointState` в топик `/joint_state`. Чтобы получить трансформы в топике `/tf`, нужно запустить пакет `robot_state_publisher`. Сначала установим его:

```
sudo apt-get install ros-humble-robot-state-publisher
sudo apt install ros-humble-sdformat-urdf
```

Далее можно включить его запуск в лаунчфайл с бриджами:

```
urdf = os.path.join(
    get_package_share_directory('jetbot_ros'),
    'models/jetbot/model_high_resolution.sdf')
with open(urdf, 'r') as infp:
    robot_desc = infp.read()

Node (
    package='robot_state_publisher',
    executable='robot_state_publisher',
    name='robot_state_publisher',
```

```

        output='screen',
        parameters=[{'use_sim_time': True, 'robot_description':
robot_desc}],
        arguments=[urdf]
    )

```

Дальше можно запустить лаунчфайл с бриджами и открыть RViz. В RViz выставим в качестве fixed frame `base_link` и откроем топик облака точек с лидара (оканчивается на `scan/points`). Мы увидим, что точки находятся не в горизонтальной плоскости, а в вертикальной. Чтобы это исправить, добавим еще один фрейм – `base_footprint`, и статический трансформ от него во фрейм `base_link`. Сделать это можно с помощью стандартного пакета `static_transform_publisher`, включив его запуск в тот же лаунчфайл с бриджами:

```

Node (
    package='tf2_ros',
    executable='static_transform_publisher',
    name='tf_base_footprint_to_base_link',
    arguments=["0", "0", "0", "0.7071", "0", "0", "-0.7071",
"base_footprint", "base_link"]
)

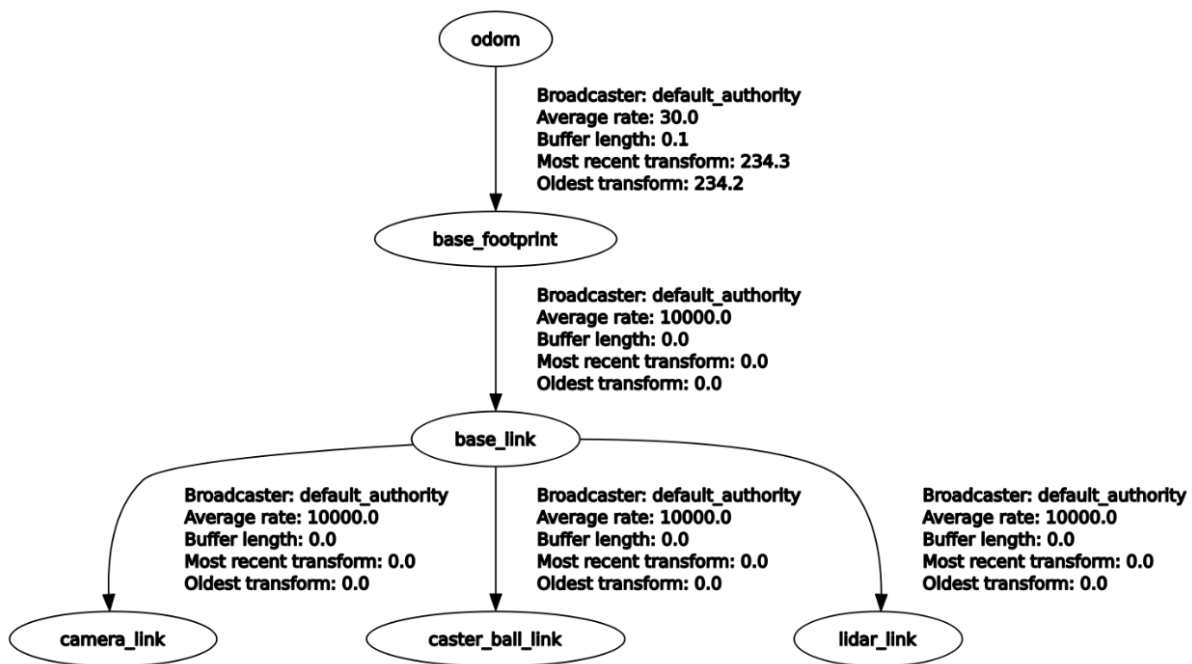
```

Осталось сделать динамический трансформ из одометрии (от фрейма `odom` до фрейма `base_footprint`). Как ни странно, в ROS2 нет готовой утилиты для преобразования одометрии в трансформ, поэтому придется написать или [навайбкодить](#) ноду для этого. Добавим ноду в пакет `jetbot_ros` и добавим ее запуск в лаунчфайл с бриджами. После этого пересоберем, перезапустим бриджи и посмотрим на дерево трансформов:

```
ros2 launch jetbot_ros_gazebo gazebo_ros_bridges.launch.py
```

```
ros2 run rqt_tf_tree rqt_tf_tree
```

Должно получиться такое дерево:



Теперь откроем RViz и выставим fixed_frame odom. Посмотрим на точки с лидара. Если они отображаются, пустим робота по кругу небольшого радиуса с помощью телеуправления в окошке Gazebo. Посмотрим на поведение точек во фрейме odom. Они не должны двигаться, могут лишь незначительно колыхаться туда-сюда. Если они уплывают или вращаются, значит, что-то настроено неправильно.